Data Structures

BS (CS) _Fall_2025

Lab_15 Task

Learning Objectives:

1. Graphs using Adjacency List

Lab Task 1

Implement a Directed Graph Using Adjacency List

You are required to **implement a directed graph** using an **adjacency list representation**. Each vertex will be stored in a **linked list node**, and each vertex node must contain:

- The **name/label** of the vertex (e.g., char or string)
- A **pointer to another linked list** containing all of its neighbors (outgoing edges)
- A pointer to the **next vertex** in the main vertex list

You must implement the following operations:

Required Functionalities

1. **Insert a new vertex**
    - If the vertex already exists, show an appropriate message.
2. **Insert a new edge (Directed)**
    - Insert an edge from vertex u → v.
    - Both vertices must exist; otherwise handle the error properly.
3. **Update a vertex**
    - Update the name/label of an existing vertex.
    - Make sure all corresponding adjacency lists are also updated.
4. **Search a vertex**
    - Return whether the vertex exists in the graph.
5. **Delete a vertex**
    - Remove the vertex from the vertex list.
    - Remove this vertex from all adjacency lists of other vertices.
6. **Delete an edge**
    - Delete a directed edge u → v.
7. **Perform DFS Traversal**
    - Take a starting vertex from the user.
    - Perform **Depth-First Search** and print the visited order.

Constraints

- Use **linked lists only** (no vectors, no STL containers).

- The graph will be **directed**.
- Graph may be disconnected.
- Carefully handle cases like: inserting duplicates, deleting non-existing vertices/edges, or DFS from a vertex that doesn't exist.

Google Test Cases

```
TEST(GraphTest, InsertSingleVertex) {
    Graph<std::string> g;
    g.insertVertex("A");
    EXPECT_TRUE(g.searchVertex("A"));
}

TEST(GraphTest, InsertDuplicateVertex) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("A");
    EXPECT_TRUE(g.searchVertex("A"));
}

TEST(GraphTest, InsertMultipleVertices) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertVertex("C");

    EXPECT_TRUE(g.searchVertex("A"));
    EXPECT_TRUE(g.searchVertex("B"));
    EXPECT_TRUE(g.searchVertex("C"));
}

TEST(GraphTest, InsertEdgeBetweenExistingVertices) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertEdge("A", "B");

    std::string dfsRes = g.dfs("A");
    EXPECT_EQ(dfsRes, "A B");
}

TEST(GraphTest, InsertEdgeNonExistingSource) {
    Graph<std::string> g;
```

```
    g.insertVertex("B");
    g.insertEdge("A", "B");
    EXPECT_FALSE(g.searchVertex("A"));
}

TEST(GraphTest, InsertEdgeNonExistingDestination) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertEdge("A", "B");
    EXPECT_FALSE(g.searchVertex("B"));
}

TEST(GraphTest, UpdateVertexName) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.updateVertex("A", "Z");
    EXPECT_FALSE(g.searchVertex("A"));
    EXPECT_TRUE(g.searchVertex("Z"));
}

TEST(GraphTest, UpdateVertexWithEdges) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertEdge("A", "B");

    g.updateVertex("B", "X");

    std::string dfsRes = g.dfs("A");
    EXPECT_EQ(dfsRes, "AX");
}

TEST(GraphTest, SearchNonExistingVertex) {
    Graph<std::string> g;
    EXPECT_FALSE(g.searchVertex("Q"));
}

TEST(GraphTest, DeleteLeafVertex) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.deleteVertex("A");
    EXPECT_FALSE(g.searchVertex("A"));
}

TEST(GraphTest, DeleteVertexWithIncomingEdges) {
```

```
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertEdge("A", "B");

    g.deleteVertex("B");
    EXPECT_FALSE(g.searchVertex("B"));

    EXPECT_EQ(g.dfs("A"), "A");
}

TEST(GraphTest, DeleteVertexWithOutgoingEdges) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertEdge("A", "B");

    g.deleteVertex("A");
    EXPECT_FALSE(g.searchVertex("A"));
}

TEST(GraphTest, DeleteNonExistingVertex) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.deleteVertex("X");
    EXPECT_TRUE(g.searchVertex("A"));
}

TEST(GraphTest, DeleteExistingEdge) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertEdge("A", "B");

    g.deleteEdge("A", "B");

    EXPECT_EQ(g.dfs("A"), "A");
}

TEST(GraphTest, DeleteNonExistingEdge) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.deleteEdge("A", "B");
    EXPECT_TRUE(g.searchVertex("A"));
```

```
    EXPECT_TRUE(g.searchVertex("B"));
}

TEST(GraphTest, DFSOnDisconnectedGraph) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");

    EXPECT_EQ(g.dfs("A"), "A");
}

TEST(GraphTest, DFSChainGraph) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertVertex("C");
    g.insertEdge("A", "B");
    g.insertEdge("B", "C");

    EXPECT_EQ(g.dfs("A"), "ABC");
}

TEST(GraphTest, DFSMultipleBranches) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertVertex("C");
    g.insertVertex("D");

    g.insertEdge("A", "B");
    g.insertEdge("A", "C");
    g.insertEdge("B", "D");

    std::string dfsRes = g.dfs("A");
    // DFS can have multiple valid orders, but one possible:
    EXPECT_EQ(dfsRes, "ABDC");
}

TEST(GraphTest, DFSComplexGraph) {
    Graph<std::string> g;
    g.insertVertex("1");
    g.insertVertex("2");
    g.insertVertex("3");
    g.insertVertex("4");
    g.insertEdge("1", "2");
```

```
    g.insertEdge("1", "3");
    g.insertEdge("2", "4");

    std::string dfsRes = g.dfs("1");
    // Possible DFS order
    EXPECT_EQ(dfsRes, "1243");
}
```

Lab Task 2

Detect Cycle in a Directed Graph

Using the adjacency list structure implemented in **Question 1**, you must implement a function to determine whether the directed graph contains a **cycle**.

Requirements

1. Implement a function bool hasCycle() that:
    - Returns true if the graph contains **any cycle**.
    - Returns false if the graph is acyclic.
2. You must detect cycles using **DFS-based cycle detection** using:
    - A visited list/array
    - A recursion-stack list/array (or color marking if preferred)
3. You **must not** use STL containers.
4. Test against cases such as:
    - Graph with no edges
    - Graph with self-loop (e.g., A → A)
    - Graph with a simple cycle (A → B → C → A)
    - Graph with multiple components
    - Graph with a cycle in one component but not the other

Algortihm:
DFS + Recursion Stack

- Mark each node as:
    - **Unvisited**
    - **Visited in current recursion stack**
    - **Visited completely**
- If during DFS you ever reach a node that is already in the **recursion stack**, a cycle exists.

Google Test Cases:

```
TEST(GraphCycleTest, NoCycleSingleNode) {
    Graph<std::string> g;
    g.insertVertex("A");
    EXPECT_FALSE(g.hasCycle());
}

TEST(GraphCycleTest, SelfLoopCycle) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertEdge("A", "A");
    EXPECT_TRUE(g.hasCycle());
}

TEST(GraphCycleTest, SimpleCycleABC) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertVertex("C");

    g.insertEdge("A", "B");
    g.insertEdge("B", "C");
    g.insertEdge("C", "A");

    EXPECT_TRUE(g.hasCycle());
}

TEST(GraphCycleTest, NoCycleLinear) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertVertex("C");

    g.insertEdge("A", "B");
    g.insertEdge("B", "C");

    EXPECT_FALSE(g.hasCycle());
}

TEST(GraphCycleTest, ComponentWithCycle) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertVertex("C");
```

```
    g.insertVertex("D");

    g.insertEdge("A", "B");
    g.insertEdge("B", "C");
    g.insertEdge("C", "A");

    g.insertEdge("D", "A");

    EXPECT_TRUE(g.hasCycle());
}

TEST(GraphCycleTest, MultipleComponentsAcyclic) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertVertex("X");
    g.insertVertex("Y");

    g.insertEdge("A", "B");
    g.insertEdge("X", "Y");

    EXPECT_FALSE(g.hasCycle());
}

TEST(GraphCycleTest, DeepBackEdgeCycle) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertVertex("C");
    g.insertVertex("D");
    g.insertVertex("E");

    g.insertEdge("A", "B");
    g.insertEdge("B", "C");
    g.insertEdge("C", "D");
    g.insertEdge("D", "B"); // back edge → cycle
    g.insertEdge("C", "E");

    EXPECT_TRUE(g.hasCycle());
}

TEST(GraphCycleTest, ComplexGraphNoCycle) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
```

```
    g.insertVertex("C");
    g.insertVertex("D");

    g.insertEdge("A", "B");
    g.insertEdge("A", "C");
    g.insertEdge("C", "D");

    EXPECT_FALSE(g.hasCycle());
}

TEST(GraphCycleTest, StarShapeNoCycle) {
    Graph<std::string> g;
    g.insertVertex("A");
    g.insertVertex("B");
    g.insertVertex("C");
    g.insertVertex("D");

    g.insertEdge("A", "B");
    g.insertEdge("A", "C");
    g.insertEdge("A", "D");

    EXPECT_FALSE(g.hasCycle());
}

TEST(GraphCycleTest, TreeStructureNoCycle) {
    Graph<std::string> g;
    g.insertVertex("1");
    g.insertVertex("2");
    g.insertVertex("3");
    g.insertVertex("4");
    g.insertVertex("5");

    g.insertEdge("1", "2");
    g.insertEdge("1", "3");
    g.insertEdge("2", "4");
    g.insertEdge("2", "5");

    EXPECT_FALSE(g.hasCycle());
}
```